

0.1.1. Afficher la lettre A — de base, ailleurs

Le même A qu'au 0.1, mais au milieu de l'écran : `texte(10, 8, "A")`.

1. Ce qu'il faut comprendre

C'est le **0.1**, avec un seul changement : la **place** du A. Il n'est plus en (0, 0), en haut à gauche, mais en **(10, 8)**, au milieu de l'écran.

Les deux premiers nombres de `texte` sont la colonne (de 0 à 19, de gauche à droite) et la ligne (de 0 à 17, de haut en bas). Le milieu de l'écran est vers la colonne 10 et la ligne 8.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.1.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **203 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         18 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles - 1 lettre sur 43
"A"            1 o ← texte() - un texte gravé
```

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte> // écrit un texte à l'écran
```

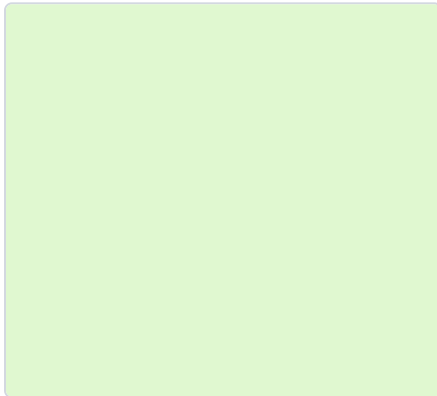
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le changement : la place. (0, 0) → (10, 8).
    //
    //  texte(10, 8, "A");
    //      |  |
    //      |  +-- ligne 8   : vers le milieu, de haut en bas (0 à 17)
    //      +----- colonne 10 : vers le milieu, de gauche à droite (0 à 19)
    texte(10, 8, "A");

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A est au milieu, en (10, 8)
- ✓ et plus en haut à gauche

Ensuite : **0.1.2. Afficher la lettre A — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++